Demo: Catch Smart Glasses If You Scan: Robot-Controlled vs. UI-Guided

Hanting Ye, Tianyi Hu, Valiantsin Kasabrukhau, Sulaiman Khydyr uulu, Maria Gorlatova
Duke University

CCS Concepts

• Human-centered computing → Interactive systems and tools; **• Security and privacy → Mobile platform security**.

Keywords

Smart Glasses Scanning and Detection, Robot Arm, UI Guidance

1 Abstract

As smart glasses become increasingly lightweight and suitable for all-day wearables, they show strong potential to serve as the next-generation mobile platforms that provide an AR interface bridging human experience and AI perception. However, a serious concern about its misuse has recently been reported [2] and has attracted public attention: smart glasses can be used as cheating tools in competitions and exams when they are indistinguishable from regular glasses. Our previous work [3] introduces a non-invasive smart glasses detection solution that leverages two unique AR optical signatures found in current smart glasses: eye glow (i.e., light leakage from AR waveguide displays) and rainbow pattern (i.e., diffraction of ambient light on AR waveguide displays). In this approach, glasses need to be scanned manually using the cameras on off-the-shelf smartphones. However, it is challenging to maintain comprehensive, multi-angle coverage that guarantees consistent and accurate detection performance, as their scanning trajectories may vary significantly and are unable to capture frames containing eye glow or rainbow patterns. Inspired by this, in this demo, we introduce two glasses scanning pipelines: (1) a robot arm-based, controllable smart glasses scanning platform; and (2) a UI-guided smart glasses scanning solution.

Pipeline Setup: In (1), the robot arm enables a reproducible smart glasses scanning process, where scanning trajectories can be customized by users. The scanning platform consists of a 4-DoF robot (PincherX 100 from Trossen Robotics), a 3D-printed face mask for mounting the smart glasses, and a smartphone camera on a stand, as shown in Figure 1(a). For robot control, we use a Linux-based Beelink Mini S12 running ROS2. We developed a UI with 11 predefined basic robot arm movements, which can be combined and adjusted in both order and amplitude of each movement to mimic human head motions within ranges of 140° yaw, 120° pitch, and 80° roll [1]. In (2), we develop a detector app on a Google Pixel 3 with UI indicators to guide users during scanning, ensuring that the smart glasses are observed from a comprehensive range of angles. We deploy the Google ML Kit face detection model to identify facial landmarks and use them to calculate the spatial orientation of the user's head pose. To simplify the user guidance experience, we create a ring around the user composed of 80 short grey bars, as shown in Figure 1(b). These bars represent different facial orientations.

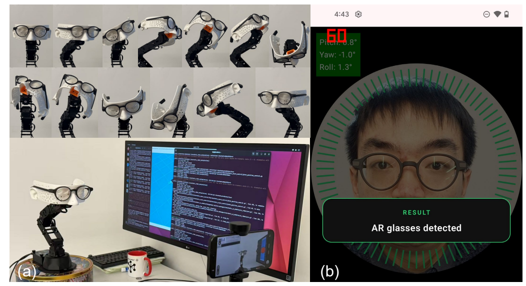

Figure 1: The setup of (a) robot-controlled glasses scanning, where scanning trajectories can be customized by users, and (b) UI-guided glasses scanning, where designed UI indicators guide users through the scanning process.

When a grey bar is covered for a certain period of time, it turns green. We also display instructional text at the bottom of the app to guide users to move toward any uncovered orientations until all the grey bars turn green.

Interactive Demonstration: Participants can experience two smart glasses scanning pipelines. At the beginning, they can either use their own glasses or choose from the provided regular or smart glasses, with or without prescription lenses. In (1), participants place the selected glasses on the 3D-printed face mask and choose several basic head movements to combine, based on their own preferences for scanning behavior. The video recorded via the smartphone camera is sent to the edge for analysis to determine whether the selected scanning behavior can correctly distinguish smart glasses from regular ones. In (2), participants use our developed app to scan their face while wearing the glasses, following UI prompts until the ring indicators change from red to green, signaling that the scanning process is complete. The recorded video is then sent to the edge for analysis to determine whether the participant is wearing smart glasses. Through these two pipelines, demo participants can observe how an unoptimized scanning behavior, such as one produced by manually controlling the robot arm, might miss key AR optical signatures on smart glasses and result in detection failures, whereas a UI-guided design enables more comprehensive scanning and a higher detection success rate.

Demonstration Video Link: https://youtu.be/6YOtyM3v9vw

Acknowledgements. This work was supported in part by NSF grants CSR-2312760, CNS-2112562, and IIS-2231975, NSF CAREER Award IIS-2046072, NSF NAIAD Award 2332744, a Cisco Research Award, a Meta Research Award, Defense Advanced Research Projects Agency Young Faculty Award HR0011-24-1-0001, and the Army Research Laboratory under Cooperative Agreement Number W911NF-23-2-0224. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Defense Advanced Research Projects Agency, the Army Research Laboratory, or the U.S. Government. This paper has been approved for public release; distribution is unlimited. No official endorsement should be inferred. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

HotMobile '26, Atlanta, GA, USA

ACM ISBN 979-8-4007-2471-8/2026/02
https://doi.org/10.1145/3789514.3796255

References

[1] Samuel Gilman et al. 1979. Measurement of head movement during auditory localization. *Behavior Research Methods & Instrumentation.*

[2] Reddit. 2025. Cheating with smart glasses? *www.reddit.com/r/Professors/comments/1kgguam/cheating_with_smart_glasses/*.

[3] Hanting Ye, Tianyi Hu, and Maria Gorlatova. 2026. No guide, no cheat: detecting smart glasses via ar optical signatures. In *Proc. HotMobile.*